

Pygame Codebase Dissection

1. Executive Summary

This project is a small 2D arcade space shooter built with Pygame. The player controls a ship, moves with WASD, fires shots with Space, destroys asteroids for points, and loses lives when the ship collides with asteroids (`telas.py::TelaJogo.update`, `sprites.py::Nave.update`, `sprites.py::Tiro.update`, `sprites.py::Asteroides.update`). The explicit lose condition is `vidas <= 0`, which returns the transient state value 2 and causes the main loop to show the game-over screen (`telas.py::TelaJogo.update`, `jogo.py::game_loop`). No win condition is present in the provided code. The tech stack is Python with Pygame, PNG images, WAV/Ogg audio, and a TTF font; no dependency manifest or Python version marker is present, and the local environment did not expose an installed Pygame version. Execution starts from top-level code in `jogo.py`, which calls `pygame.init()`, constructs the screen objects, and calls `game_loop(telas)` (`jogo.py::__main__`).

The project is split into a small entry script, screen classes, and sprite classes. `TelaJogo` owns most gameplay behavior: input, spawning, collisions, scoring, difficulty, audio startup, and drawing (`telas.py::TelaJogo.__init__`, `telas.py::TelaJogo.update`, `telas.py::TelaJogo.desenha`). There is no separate game engine object or configuration module; the code uses hardcoded numeric state IDs and hardcoded constants throughout (`jogo.py::game_loop`, `jogo.py::__main__`, `telas.py::TelaJogo.update`, `sprites.py::Nave.__init__`, `sprites.py::Asteroides.__init__`).

2. Project Structure

Path	Purpose	Evidence
<code>jogo.py</code>	Entry point, Pygame initialization, screen dictionary, main loop	<code>jogo.py::__main__</code> , <code>jogo.py::game_loop</code>
<code>telas.py</code>	Screen classes for initial menu, gameplay, and game over	<code>telas.py::TelaInicial</code> , <code>telas.py::TelaJogo</code> , <code>telas.py::TelaGameOver</code>
<code>sprites.py</code>	Player, asteroid, bullet, and explosion sprite classes	<code>sprites.py::Nave</code> , <code>sprites.py::Asteroides</code> , <code>sprites.py::Tiro</code> , <code>sprites.py::Explosao</code>
<code>funcoes.py</code>	Empty module imported by <code>jogo.py</code> ; no observed runtime behavior	<code>funcoes.py</code> , <code>jogo.py::__main__</code>
<code>README.md</code>	Course/project placeholder documentation and Pygame install note	<code>README.md</code>
<code>assets/img/</code>	Runtime PNG images for ship, asteroid, bullet, background, explosion frames	<code>sprites.py::Nave.__init__</code> , <code>sprites.py::Asteroides.__init__</code> , <code>sprites.py::Tiro.__init__</code> , <code>sprites.py::Explosao.__init__</code> , <code>telas.py::TelaJogo.__init__</code>
<code>assets/snd/</code>	Runtime music/shot sound plus unused explosion sounds	<code>telas.py::TelaJogo.__init__</code>
<code>assets/font/</code>	Runtime font plus font metadata/license files	<code>telas.py::TelaInicial.__init__</code> , <code>telas.py::TelaJogo.__init__</code> , <code>telas.py::TelaGameOver.__init__</code>

Approximate Python LOC:

Module	Approx. LOC
<code>jogo.py</code>	28
<code>sprites.py</code>	168
<code>telas.py</code>	267
<code>funcoes.py</code>	0

Third-party imports:

Import	Used for	Evidence
<code>pygame</code>	Initialization, window/display, events, sprites, images, sounds, fonts, timing, drawing	<code>jogo.py::__main__</code> , <code>jogo.py::game_loop</code> , <code>telas.py::*</code> , <code>sprites.py::*</code>
<code>random</code>	Random stars, asteroid spawn positions, asteroid sizes, asteroid movement	<code>telas.py::TelaJogo.__init__</code> , <code>sprites.py::Asteroides.__init__</code>

Internal dependencies:

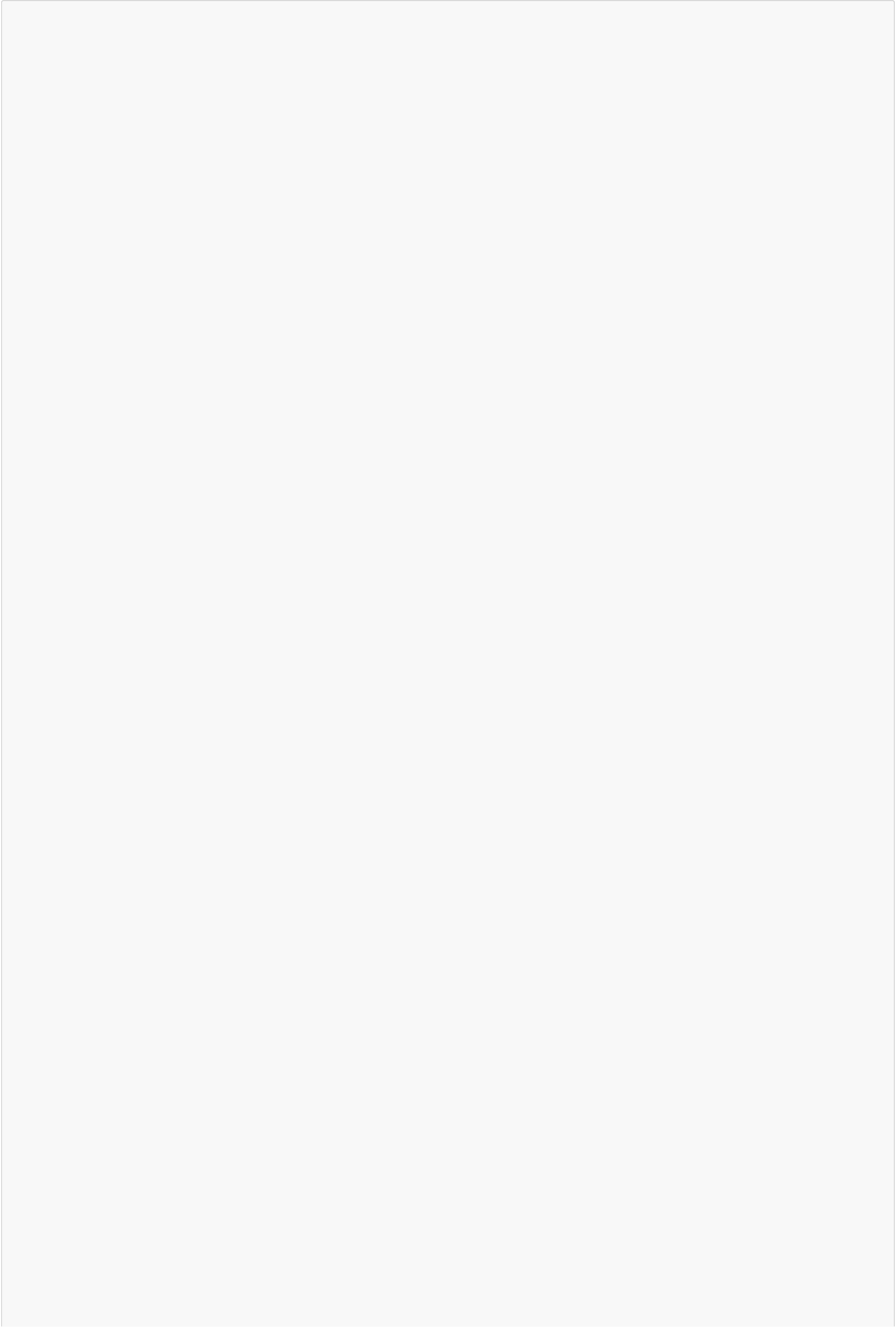
Dependency	Meaning	Evidence
<code>jogo.py -> telas.py</code>	Entry point imports all screen classes	<code>jogo.py::__main__</code>
<code>jogo.py -> funcoes.py</code>	Imports an empty module; no observed usage	<code>jogo.py::__main__</code> , <code>funcoes.py</code>

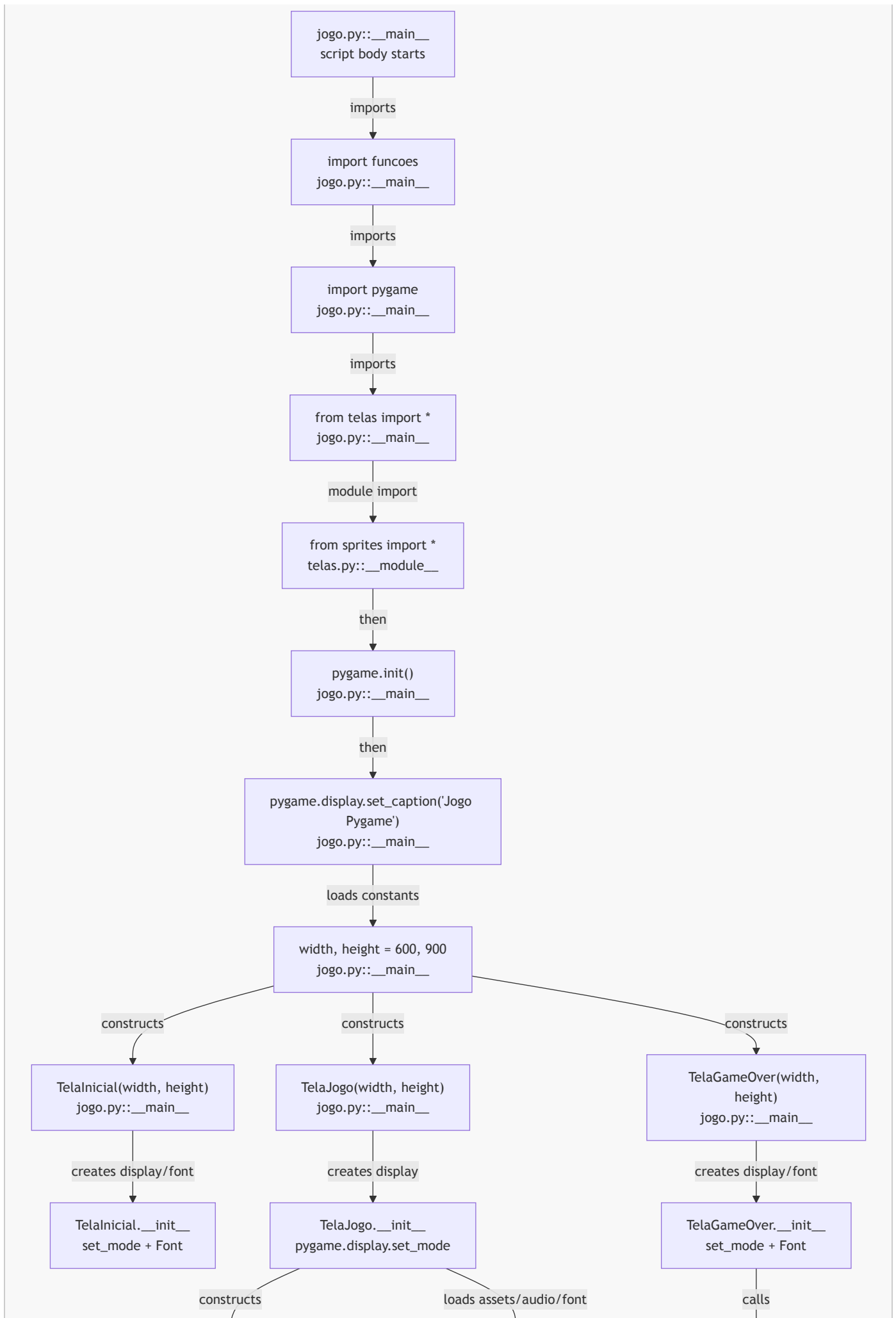
Dependency	Meaning	Evidence
telas.py -> sprites.py	Gameplay screen constructs and calls sprite classes	telas.py::__module__, telas.py::TelaJogo.__init__, telas.py::TelaJogo.update
sprites.py	No internal module dependencies	sprites.py::__module__

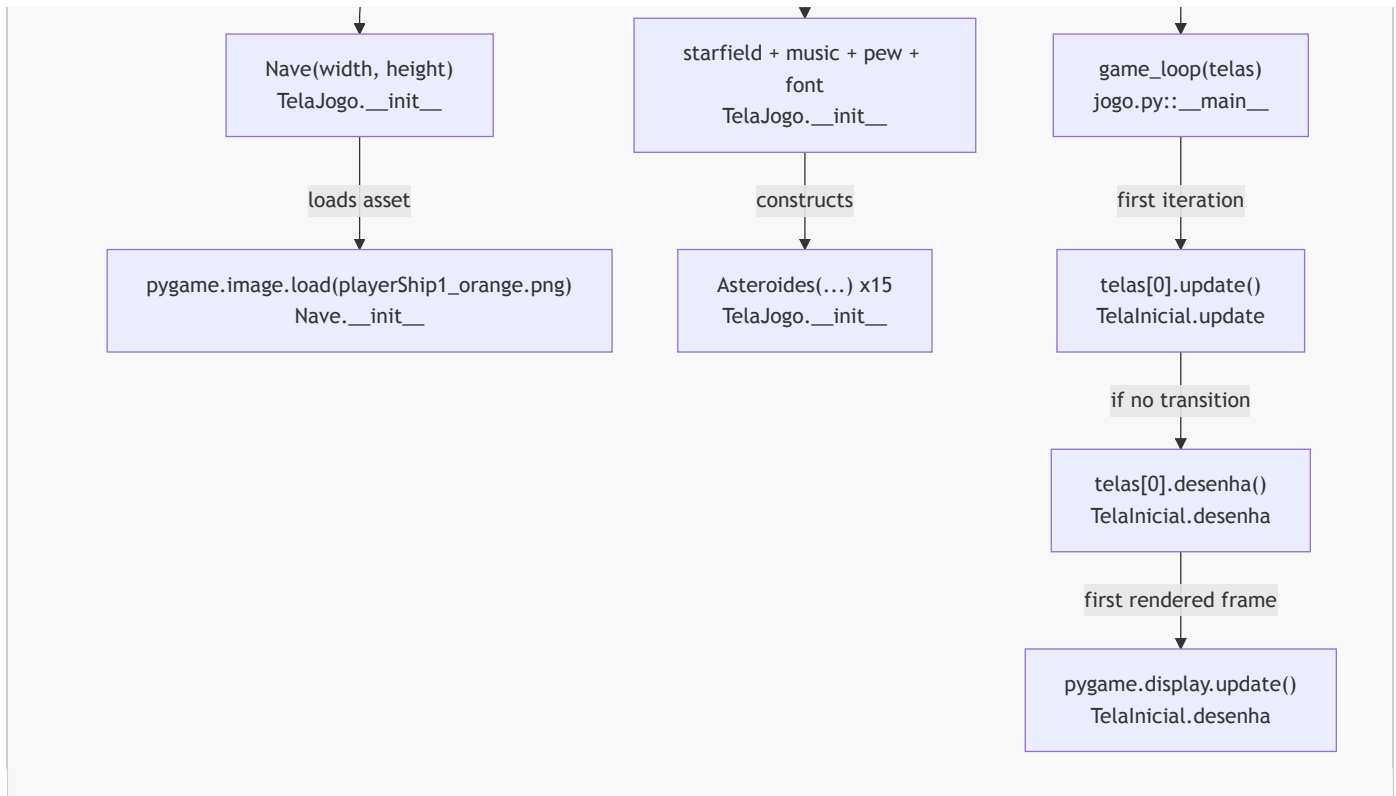
Config/constants module: not found. Window size, FPS cap, lives, asset paths, movement speeds, score rules, colors, and spawn counts are hardcoded in normal code paths (jogo.py::__main__, jogo.py::game_loop, telas.py::TelaJogo.__init__, telas.py::TelaJogo.update, sprites.py::Nave.__init__, sprites.py::Asteroides.__init__, sprites.py::Tiro.__init__, sprites.py::Explosao.__init__).

3. Runtime Architecture

3.1 Startup Path — Boot Execution Flowchart

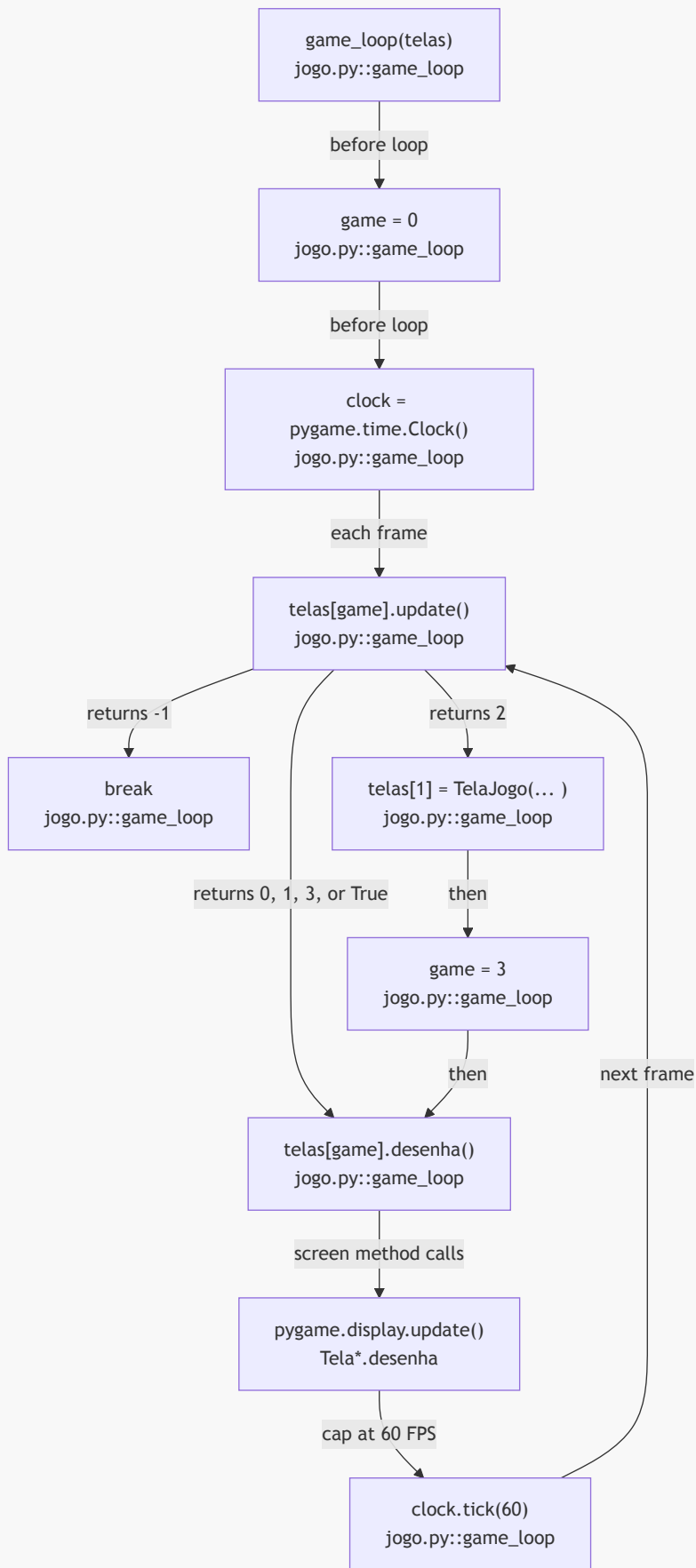






1. The script starts in `jogo.py` and imports `funcoes`, `pygame`, and every public name from `telas.py`; importing `telas.py` imports sprite classes from `sprites.py` (`jogo.py::__main__`, `telas.py::__module__`).
2. Pygame is initialized, then the window caption is set to "Jogo Pygame" (`jogo.py::__main__`).
3. The window size is hardcoded as `600 x 900` (`jogo.py::__main__`).
4. The `telas` dictionary is built with `TelaInicial`, `TelaJogo`, a placeholder string for state 2, and `TelaGameOver` (`jogo.py::__main__`).
5. All three screen objects are constructed before the loop starts; each screen constructor calls `pygame.display.set_mode((width, height))` (`telas.py::TelaInicial.__init__`, `telas.py::TelaJogo.__init__`, `telas.py::TelaGameOver.__init__`).
6. `TelaJogo.__init__` creates the player ship, loads gameplay assets, creates star data, creates 15 asteroids, initializes groups, lives, score, and level (`telas.py::TelaJogo.__init__`, `sprites.py::Nave.__init__`, `sprites.py::Asteroides.__init__`).
7. `game_loop(telas)` starts at state 0, so the first normal update/draw goes through `TelaInicial.update()` and `TelaInicial.desenha()` (`jogo.py::game_loop`, `telas.py::TelaInicial.update`, `telas.py::TelaInicial.desenha`).

3.2 Main Loop Flowchart — Per-Frame Execution Path

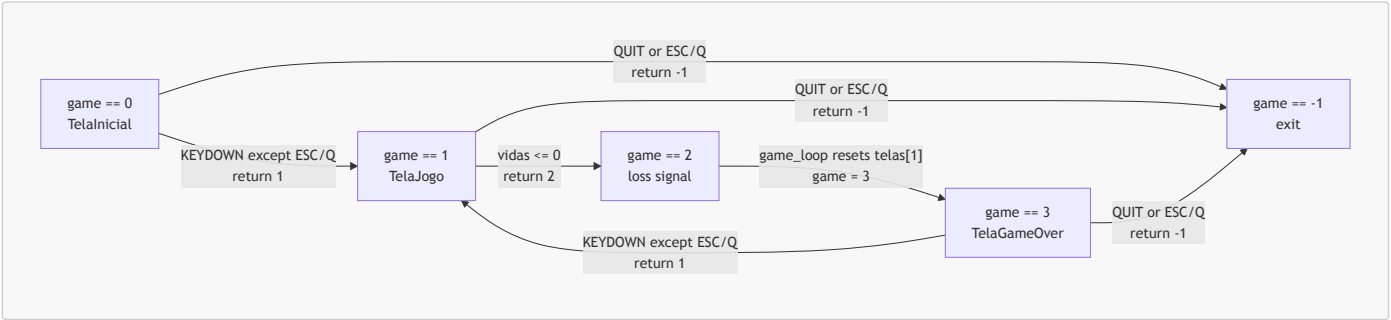


The main loop uses `clock.tick(60)`, so the intended frame cap is 60 FPS (`jogo.py::game_loop`). There is no central `delta_time` passed through the loop. Instead, `Nave.update`, `Asteroides.update`, `Tiro.update`, and `TelaJogo.desenha` each compute $1000 / (t1 - t0)$ from `pygame.time.get_ticks()` (`sprites.py::Nave.update`, `sprites.py::Asteroides.update`, `sprites.py::Tiro.update`, `telas.py::TelaJogo.desenha`). Menu, gameplay, and game-over behavior are selected by numeric state values stored in `game` (`jogo.py::game_loop`, `telas.py::TelaInicial.update`, `telas.py::TelaJogo.update`, `telas.py::TelaGameOver.update`).

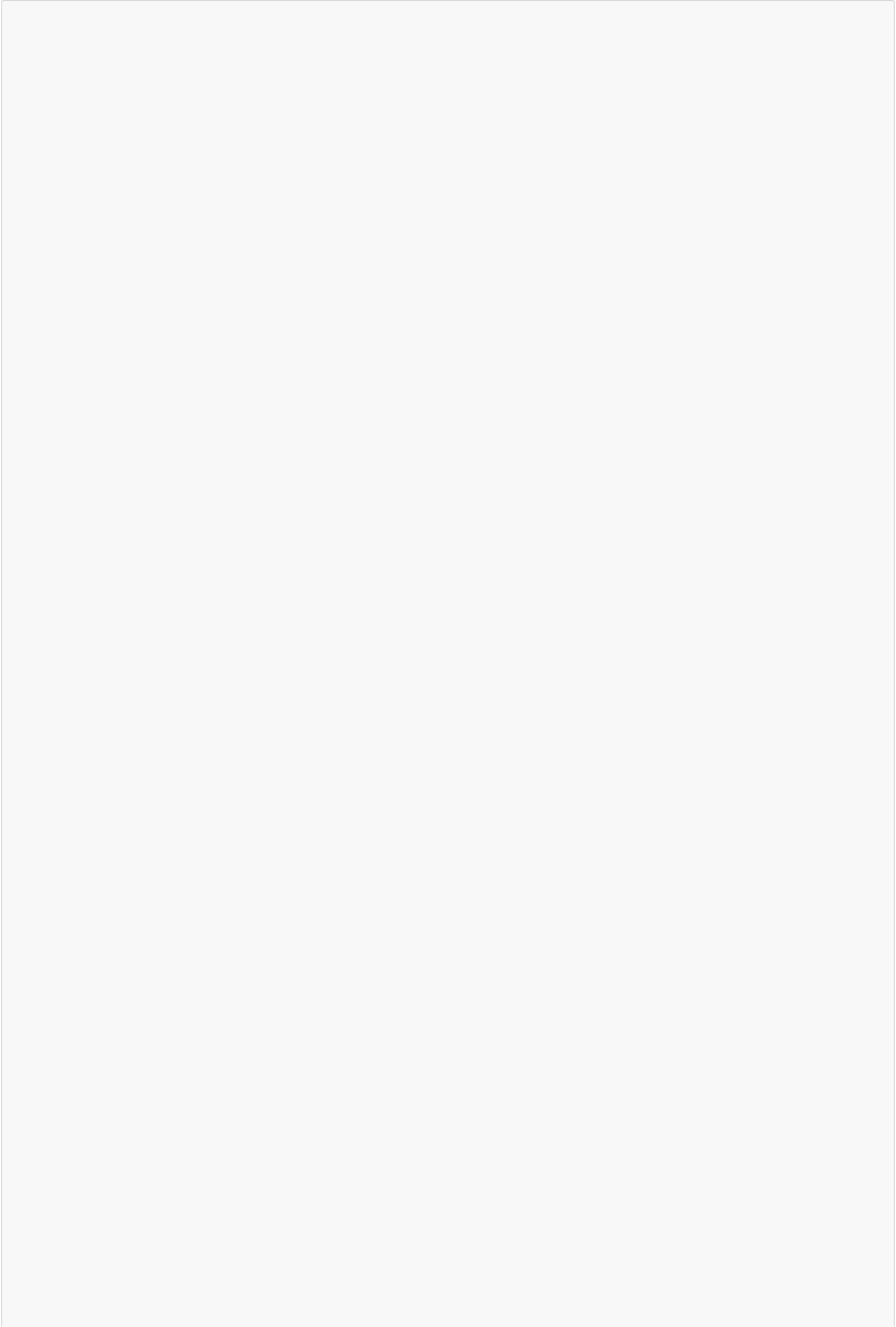
3.3 Game States & Transitions

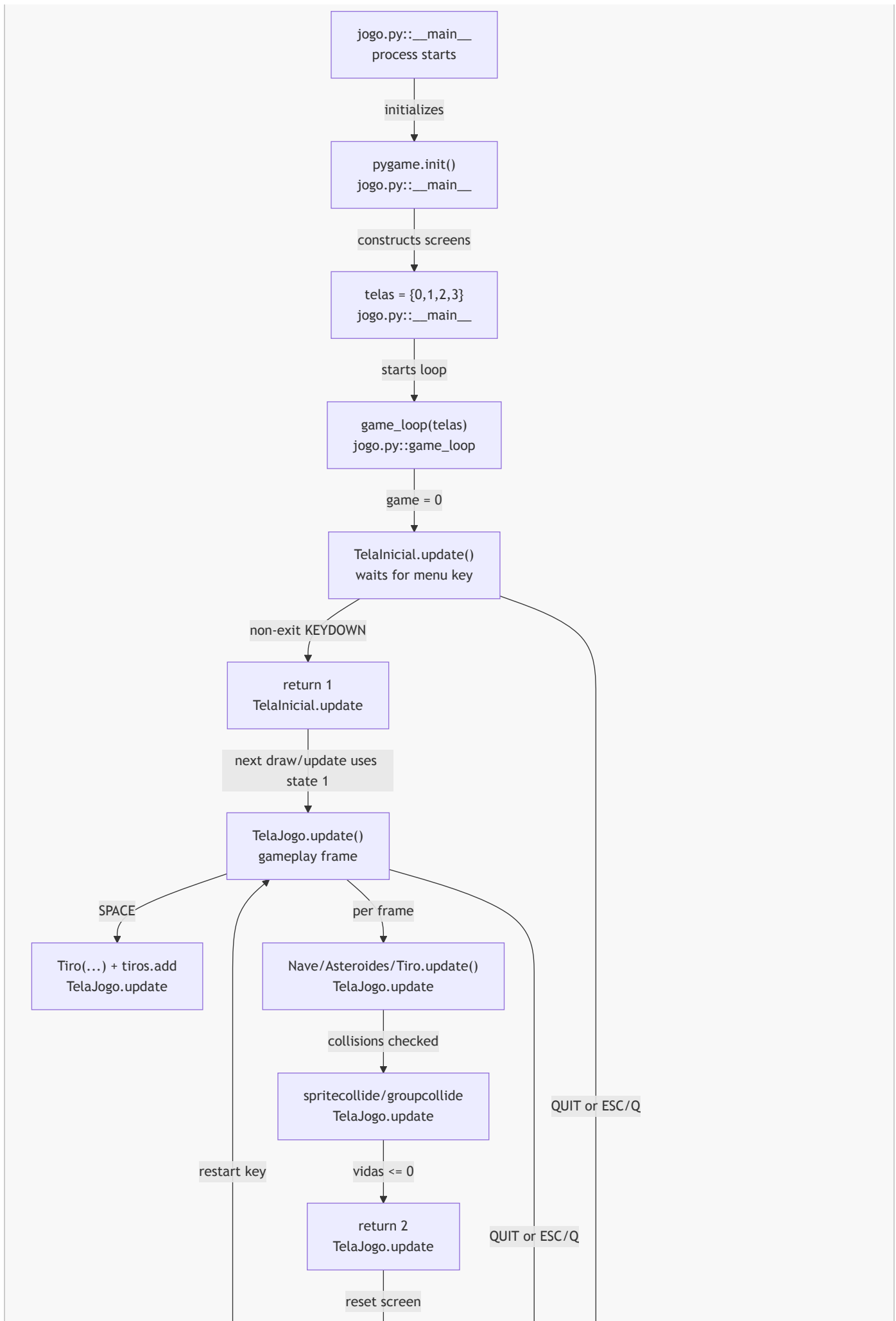
The code has no formal state enum. The following state machine is inferred from `game`, dictionary keys, and return values (`jogo.py::game_loop`, `jogo.py::__main__`).

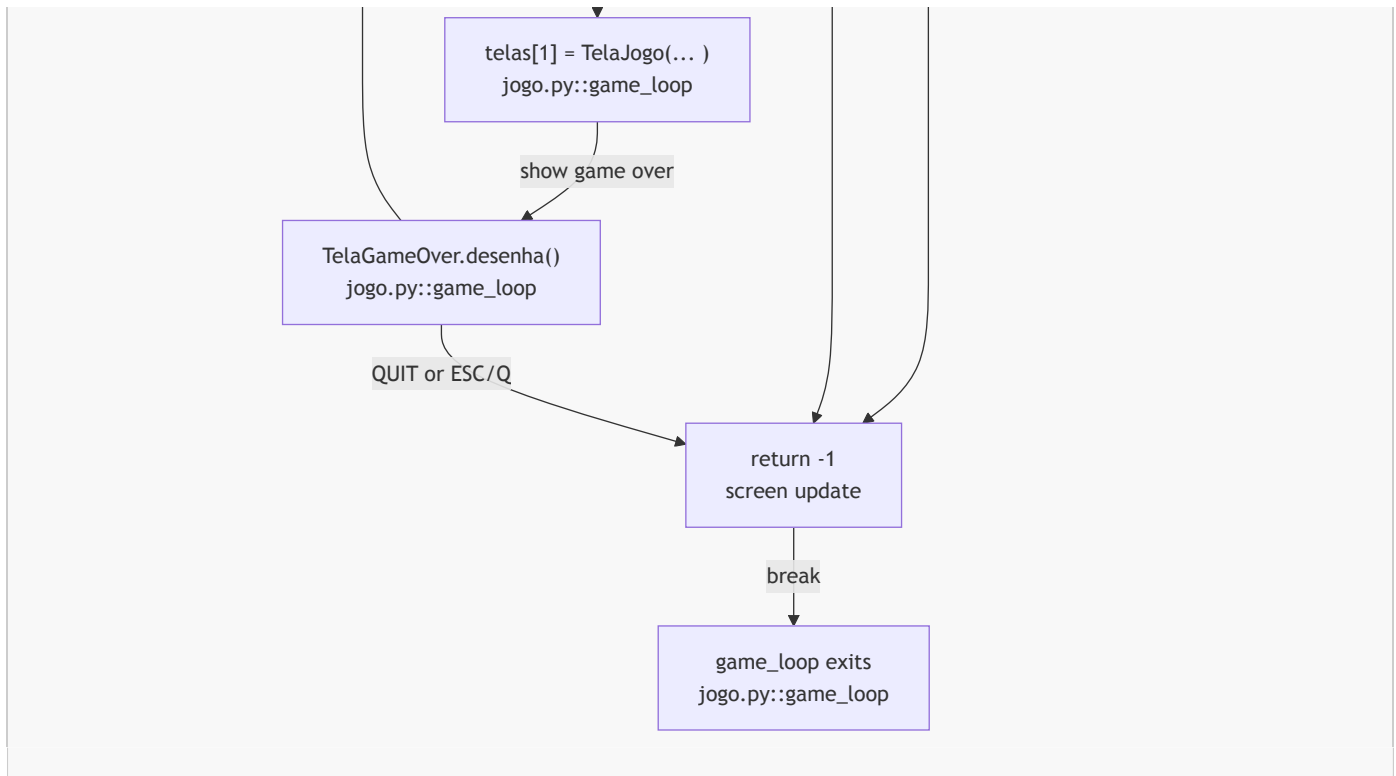
State	How code represents it	Entered from	Exited to	Trigger	Evidence
Initial menu	<code>game == 0</code> , <code>telas[0]</code> is <code>TelaInicial</code>	Process start	Gameplay or exit	Any <code>KEYDOWN</code> except <code>ESC/Q</code> returns 1; <code>QUIT</code> , <code>ESC</code> , or <code>Q</code> returns -1	<code>jogo.py::__main__</code> , <code>telas.py::TelaInicial.update</code>
Gameplay	<code>game == 1</code> ; normal <code>TelaJogo.update</code> returns <code>True</code> , which behaves like key 1 in Python dictionaries	Initial menu or game over	Game over or exit	<code>vidas <= 0</code> returns 2; <code>QUIT</code> , <code>ESC</code> , or <code>Q</code> returns -1	<code>jogo.py::game_loop</code> , <code>telas.py::TelaJogo.update</code>
Player-lost transient	<code>game == 2</code> ; <code>telas[2]</code> is an empty string but is intercepted before drawing	Gameplay	Game over	<code>TelaJogo.update</code> returns 2; <code>game_loop</code> resets <code>telas[1]</code> and sets <code>game = 3</code>	<code>jogo.py::__main__</code> , <code>jogo.py::game_loop</code> , <code>telas.py::TelaJogo.update</code>
Game over	<code>game == 3</code> , <code>telas[3]</code> is <code>TelaGameOver</code>	Player-lost transient	Gameplay or exit	Any <code>KEYDOWN</code> except <code>ESC/Q</code> returns 1; <code>QUIT</code> , <code>ESC</code> , or <code>Q</code> returns -1	<code>jogo.py::__main__</code> , <code>jogo.py::game_loop</code> , <code>telas.py::TelaGameOver.update</code>
Exit	<code>game == -1</code> return value	Any screen	Loop terminates	Active screen returns -1	<code>jogo.py::game_loop</code> , <code>telas.py::TelaInicial.update</code> , <code>telas.py::TelaJogo.update</code> , <code>telas.py::TelaGameOver.update</code>



3.4 End-to-End Execution Path Flowchart







This is the normal reading path for the codebase: startup builds all screens, `game_loop` starts in `TelaInicial`, a key moves into `TelaJogo`, and gameplay repeats until the player loses all lives or exits. Game over reconstructs `TelaJogo` before drawing `TelaGameOver`, so the next restart uses a fresh gameplay screen (`jogo.py::game_loop`).

4. Subsystem Inventory

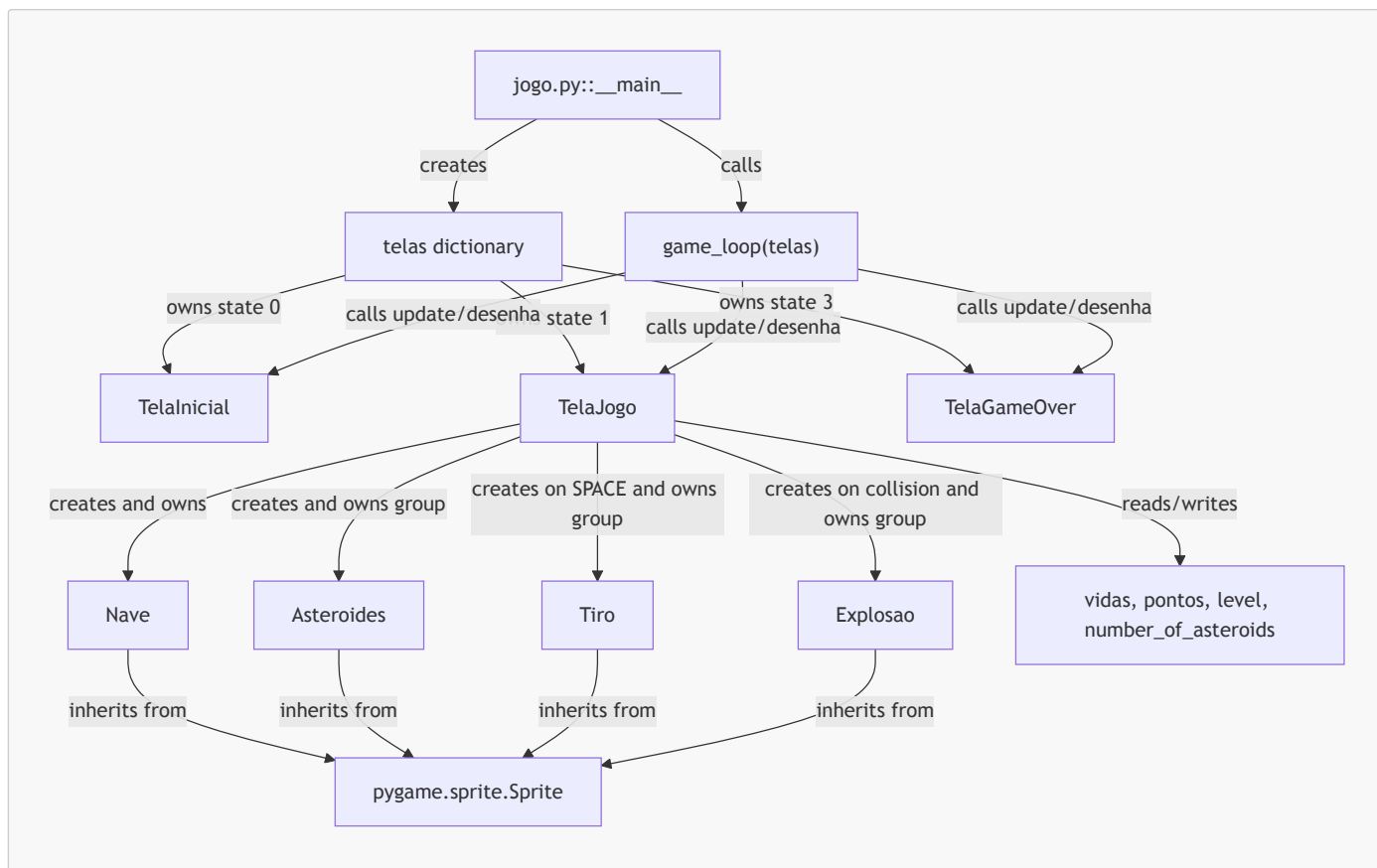
Subsystem	Exists?	Description	Main files/classes/functions
Input	Yes	Event-based input using <code>pygame.event.get()</code> . Menus react to any non-exit key; gameplay uses <code>W/A/S/D</code> , <code>Space</code> , <code>ESC/Q</code> , and window quit.	<code>telas.py::TelaInicial.update</code> , <code>telas.py::TelaJogo.update</code> , <code>telas.py::TelaGameOver.update</code> , <code>sprites.py::Nave.keydown_event</code> , <code>sprites.py::Nave.keyup_event</code>
Rendering	Yes	Each screen has <code>desenha()</code> . Gameplay draws background, stars, HUD text, bullets, ship, asteroids, explosions, and score.	<code>telas.py::TelaInicial.desenha</code> , <code>telas.py::TelaJogo.desenha</code> , <code>telas.py::TelaGameOver.desenha</code> , <code>sprites.py::* .draw</code>
Animation	Partial	Explosions animate through 9 frames at 50 ms intervals. Ship, asteroid, and bullet use static images.	<code>sprites.py::Explosao.__init__</code> , <code>sprites.py::Explosao.update</code> , <code>telas.py::TelaJogo.desenha</code>
Physics/movement	Yes	Manual rectangle movement using inverse elapsed-time calculations. No gravity, friction, acceleration, or physics engine.	<code>sprites.py::Nave.update</code> , <code>sprites.py::Asteroides.update</code> , <code>sprites.py::Tiro.update</code>
Collision	Yes	Pygame sprite collision helpers detect ship-asteroid and bullet-asteroid overlaps.	<code>telas.py::TelaJogo.update</code>
AI/behavior	Minimal	Asteroids spawn randomly and drift; no decision-making AI.	<code>sprites.py::Asteroides.__init__</code> , <code>sprites.py::Asteroides.update</code> , <code>telas.py::TelaJogo.update</code>
Audio	Yes	Background music loops once gameplay starts; shot sound plays on <code>Space</code> . Explosion audio files exist but are unused.	<code>telas.py::TelaJogo.__init__</code> , <code>telas.py::TelaJogo.update</code>
UI/HUD/menu	Yes	Initial menu, game-over screen, lives hearts, FPS text, and points text.	<code>telas.py::TelaInicial</code> , <code>telas.py::TelaGameOver</code> , <code>telas.py::TelaJogo.desenha</code>
Level loading/content	Not formal	No map files or scene loaders. Difficulty increases by score-derived <code>level</code> and asteroid count.	<code>telas.py::TelaJogo.__init__</code> , <code>telas.py::TelaJogo.update</code>
Persistence/save/load	Not found	No file I/O, save/load, high-score, or profile code.	All provided Python modules
Debug/dev tooling	Minimal	FPS text is drawn during gameplay.	<code>telas.py::TelaJogo.desenha</code>

5. Code Structure Map (Plain-Language, Non-UML)

5.1 Component Table

Name	Kind	Lives in	What it does	Owns / holds	Talks to	Called by	Evidence
<code>game_loop(telas)</code>	function	<code>jogo.py</code>	Runs the active screen update/draw loop and handles exit/game-over transitions.	Local <code>game</code> , local <code>clock</code>	Active screen objects, <code>TelaJogo</code> , <code>pygame.time.Clock</code>	<code>jogo.py::__main__</code>	<code>jogo.py::game_loop</code>
<code>telas</code>	module-level group	<code>jogo.py</code>	Maps state IDs to screen objects.	<code>TelaInicial</code> , <code>TelaJogo</code> , placeholder <code>'</code> , <code>TelaGameOver</code>	Screen instances	<code>game_loop</code>	<code>jogo.py::__main__</code> , <code>jogo.py::game_loop</code>
<code>TelaInicial</code>	class	<code>telas.py</code>	Draws and updates the start screen.	<code>width</code> , <code>height</code> , <code>window</code> , <code>fonte</code>	<code>pygame.display</code> , <code>pygame.font</code> , <code>pygame.event</code>	<code>jogo.py::__main__</code> , <code>game_loop</code>	<code>telas.py::TelaInic</code>
<code>TelaJogo</code>	class	<code>telas.py</code>	Owns main gameplay state, input, entities, collision, scoring, audio, and rendering.	<code>nave</code> , <code>asteroides</code> , <code>tiros</code> , <code>explosoes</code> , <code>vidas</code> , <code>pontos</code> , <code>level</code> , assets	<code>Nave</code> , <code>Asteroides</code> , <code>Tiro</code> , <code>Explosao</code> , Pygame APIs	<code>jogo.py::__main__</code> , <code>jogo.py::game_loop</code>	<code>telas.py::TelaJogo</code>
<code>TelaGameOver</code>	class	<code>telas.py</code>	Draws and updates the game-over/restart screen.	<code>width</code> , <code>height</code> , <code>window</code> , <code>fonte</code>	<code>pygame.display</code> , <code>pygame.font</code> , <code>pygame.event</code>	<code>jogo.py::__main__</code> , <code>game_loop</code>	<code>telas.py::TelaGame</code>
<code>Nave</code>	class	<code>sprites.py</code>	Represents the player ship and handles movement state.	<code>image</code> , <code>rect</code> , <code>vel</code> , <code>key_state</code> , <code>t0</code>	Pygame image/transform/time APIs	<code>TelaJogo.__init__</code> , <code>TelaJogo.update</code> , <code>TelaJogo.desenha</code>	<code>sprites.py::Nave</code> , <code>telas.py::TelaJogo</code>
<code>Asteroides</code>	class	<code>sprites.py</code>	Represents a falling asteroid obstacle.	<code>imagem</code> , <code>rect</code> , <code>vel_x</code> , <code>vel_y</code> , <code>t0</code>	<code>random</code> , Pygame image/transform/time APIs	<code>TelaJogo.__init__</code> , <code>TelaJogo.update</code>	<code>sprites.py::Asteroides</code> , <code>telas.py::TelaJogo</code>
<code>Tiro</code>	class	<code>sprites.py</code>	Represents a player shot.	<code>image</code> , <code>rect</code> , <code>vel</code> , <code>t0</code>	Pygame image/transform/time APIs	<code>TelaJogo.update</code> , <code>TelaJogo.desenha</code>	<code>sprites.py::Tiro</code> , <code>telas.py::TelaJogo</code>
<code>Explosao</code>	class	<code>sprites.py</code>	Represents a temporary explosion animation.	<code>frames</code> , <code>current_frame</code> , <code>image</code> , <code>rect</code> , <code>frame_rate</code> , <code>finished</code>	Pygame image/transform/time APIs	<code>TelaJogo.update</code> , <code>TelaJogo.desenha</code>	<code>sprites.py::Explosao</code> , <code>telas.py::TelaJogo</code>
<code>funcoes</code>	module	<code>funcoes.py</code>	Empty imported module; no behavior.	Nothing	Nothing	<code>jogo.py::__main__</code>	<code>funcoes.py</code> , <code>jogo.py::__main__</code>

5.2 Component Relationship Flowchart



The entry script creates a dictionary of screen objects and hands it to `game_loop`. `game_loop` does not know gameplay details; it calls `update()` and `desenha()` on whichever screen ID is active. `TelaJogo` is the central gameplay component and owns the player, sprite groups, gameplay counters, audio objects, and drawing assets (`jogo.py::__main__`, `jogo.py::game_loop`, `telas.py::TelaJogo.__init__`).

5.3 Per-Class Field & Method Lists

TelaInicial

- **Purpose:** Shows the start screen and decides whether to start gameplay or exit.
- **Key fields:** `width` — int — window width; `height` — int — window height; `window` — Pygame surface — display target; `fonte` — Pygame font — text renderer.
- **Key methods:** `update()` — reads events and returns `-1`, `0`, or `1`; `desenha()` — draws menu text and updates the display.
- **Inherits from:** no custom or Pygame parent class.
- **Evidence:** `telas.py::TelaInicial.__init__`, `telas.py::TelaInicial.update`, `telas.py::TelaInicial.desenha`.

TelaJogo

- **Purpose:** Runs the active gameplay screen.
- **Key fields:** `nave` — `Nave` — player ship; `asteroides` — sprite group — active asteroids; `tiros` — sprite group — active shots; `explosoes` — sprite group — active explosion animations; `vidas` — int — current lives; `pontos` — number — score; `level` — int — difficulty level; `number_of_asteroids` — int — target asteroid count; `som_tiro` — sound — shot sound.
- **Key methods:** `update()` — handles input, movement, spawning, collisions, scoring, level updates, music start; `desenha()` — draws gameplay and HUD, updates FPS, updates explosion animations, updates display.
- **Inherits from:** no custom or Pygame parent class.
- **Evidence:** `telas.py::TelaJogo.__init__`, `telas.py::TelaJogo.update`, `telas.py::TelaJogo.desenha`.

TelaGameOver

- **Purpose:** Shows the game-over screen and decides whether to restart or exit.
- **Key fields:** `width`, `height`, `window`, `fonte`.
- **Key methods:** `update()` — reads events and returns `-1`, `1`, or `3`; `desenha()` — draws game-over/restart text and updates display.
- **Inherits from:** no custom or Pygame parent class.
- **Evidence:** `telas.py::TelaGameOver.__init__`, `telas.py::TelaGameOver.update`, `telas.py::TelaGameOver.desenha`.

Nave

- **Purpose:** Represents the player-controlled ship.
- **Key fields:** `image` — surface — scaled ship image; `rect` — Pygame rect — ship position and bounds; `vel` — float `0.1` — movement multiplier; `key_state` — dict — held movement keys; `t0` — int — previous tick timestamp.
- **Key methods:** `keydown_event(event)` — sets movement flags; `keyup_event(event)` — clears movement flags; `update()` — moves ship within bounds; `draw(surface)` — blits ship.
- **Inherits from:** `pygame.sprite.Sprite`, so it can participate in Pygame sprite groups and collision helpers.

- **Evidence:** `sprites.py::Nave.__init__`, `sprites.py::Nave.keydown_event`, `sprites.py::Nave.keyup_event`, `sprites.py::Nave.update`, `sprites.py::Nave.draw`.

Asteroides

- **Purpose:** Represents a falling asteroid.
- **Key fields:** `imagem` — surface — scaled asteroid image; `rect` — Pygame rect — position and bounds; `vel_x`, `vel_y` — floats — movement multipliers; `t0` — int — previous tick timestamp.
- **Key methods:** `update()` — moves asteroid and kills it after leaving the bottom of the screen; `draw(surface)` — blits asteroid.
- **Inherits from:** `pygame.sprite.Sprite`, so it can be stored in sprite groups and used by collision helpers.
- **Evidence:** `sprites.py::Asteroides.__init__`, `sprites.py::Asteroides.update`, `sprites.py::Asteroides.draw`.

Tiro

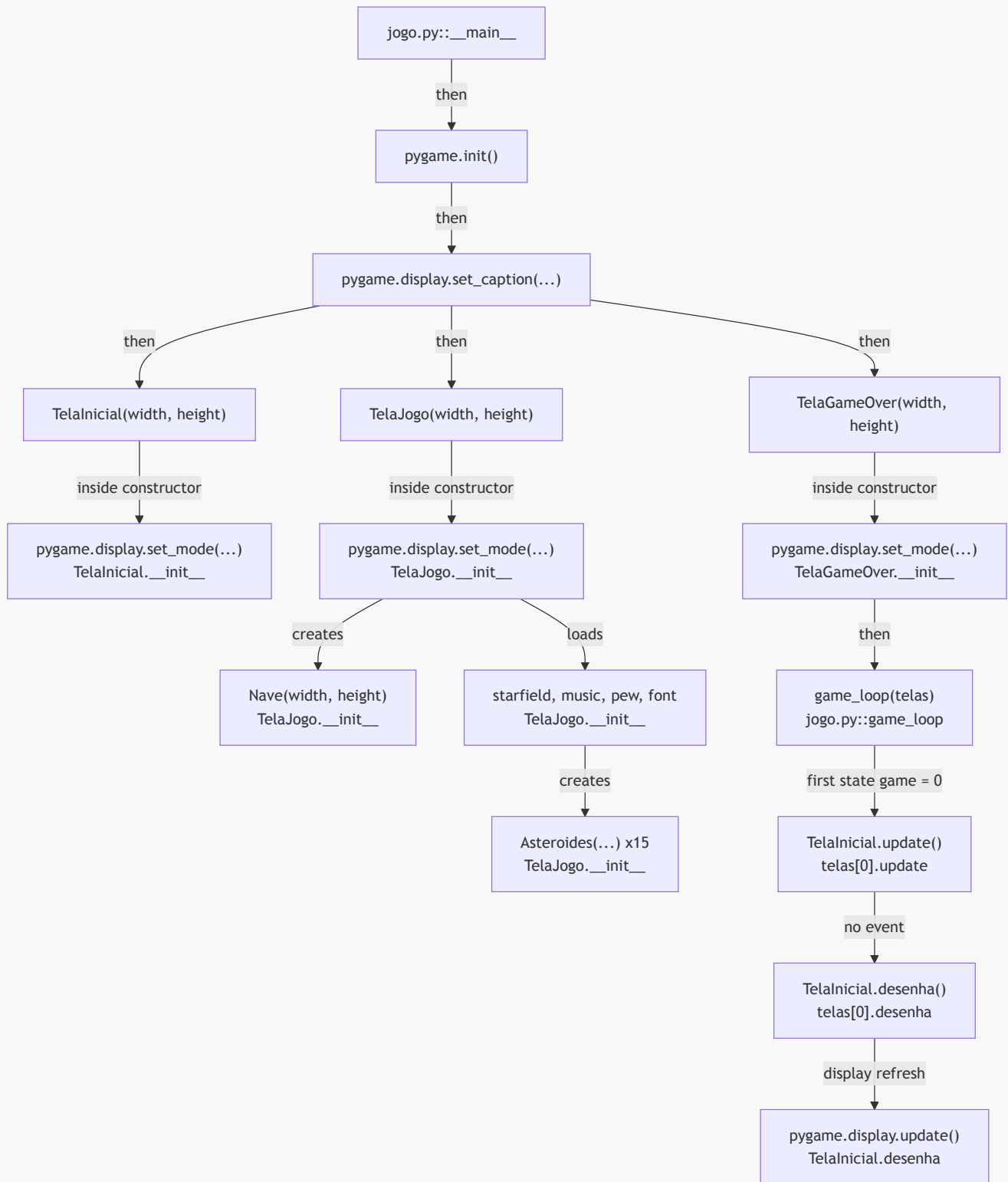
- **Purpose:** Represents a projectile fired by the player.
- **Key fields:** `image` — surface — scaled laser image; `rect` — Pygame rect — position and bounds; `vel` — float `0.3` — upward movement multiplier; `t0` — int initialized to `-1`.
- **Key methods:** `update()` — moves shot upward and kills it after leaving the top of the screen; `draw(surface)` — blits shot.
- **Inherits from:** `pygame.sprite.Sprite`, so it can be stored in sprite groups and used by collision helpers.
- **Evidence:** `sprites.py::Tiro.__init__`, `sprites.py::Tiro.update`, `sprites.py::Tiro.draw`.

Explosao

- **Purpose:** Represents a short explosion animation.
- **Key fields:** `frames` — list of surfaces — 9 explosion frames; `current_frame` — int — active frame index; `image` — surface — active frame; `rect` — Pygame rect — animation position; `frame_rate` — int `50` — milliseconds between frames; `finished` — bool — removal signal.
- **Key methods:** `update()` — advances animation frame and sets `finished`; `draw(surface)` — blits current frame.
- **Inherits from:** `pygame.sprite.Sprite`, although the code manually iterates and removes explosions instead of relying only on group update/draw.
- **Evidence:** `sprites.py::Explosao.__init__`, `sprites.py::Explosao.update`, `sprites.py::Explosao.draw`, `tela.py::TelaJogo.desenha`.

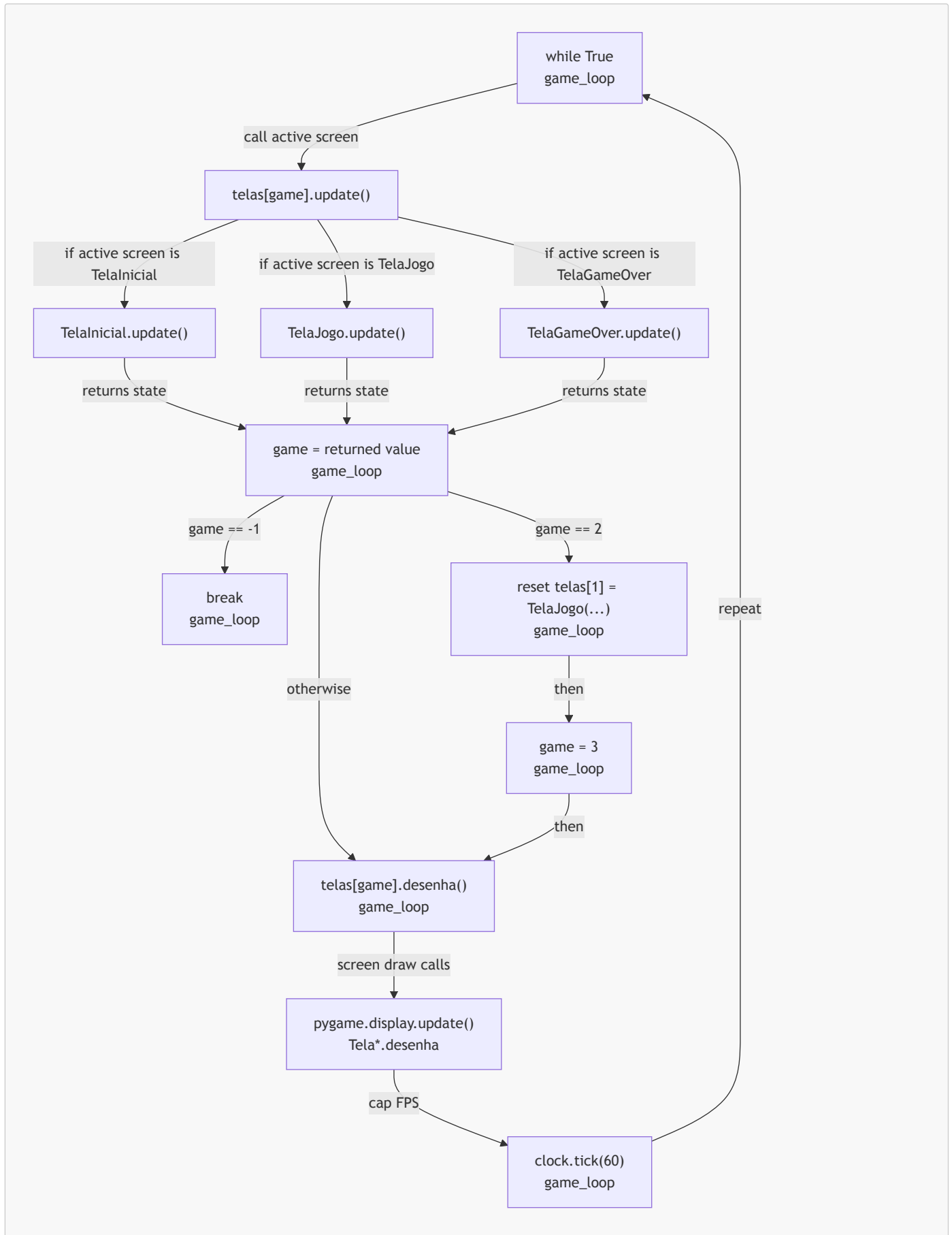
6. Execution Path Flowcharts

6.1 Game Boot Path



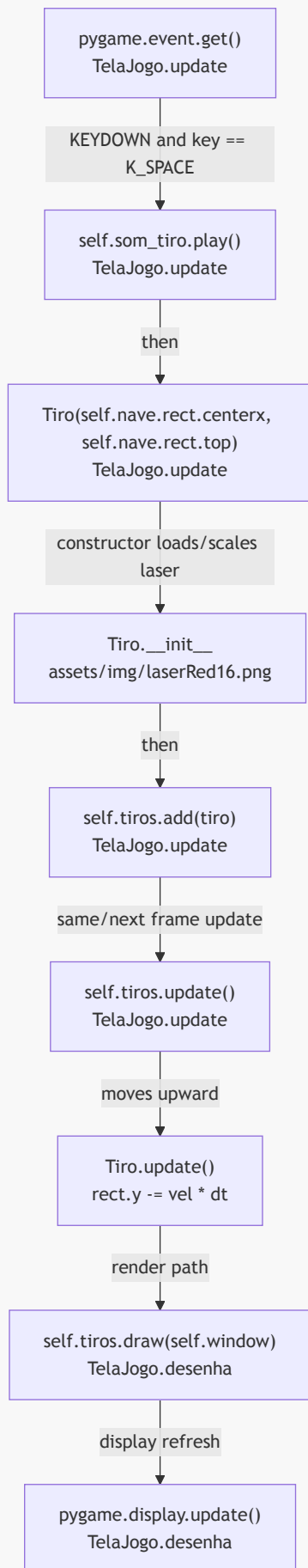
The boot path eagerly constructs all screens, including the gameplay screen and its assets, before the player starts the game. The first visible frame is the initial menu if no startup event changes the state. The code calls `pygame.display.update()`, not `pygame.display.flip()` (`telas.py::TelaInicial.desenha`, `telas.py::TelaJogo.desenha`, `telas.py::TelaGameOver.desenha`).

6.2 One Full Frame Path



Each frame delegates behavior to the current screen object. Only `game_loop` knows about the special `game == 2` loss signal. There is no pause state or skipped update branch in the provided code (`jogo.py::game_loop`).

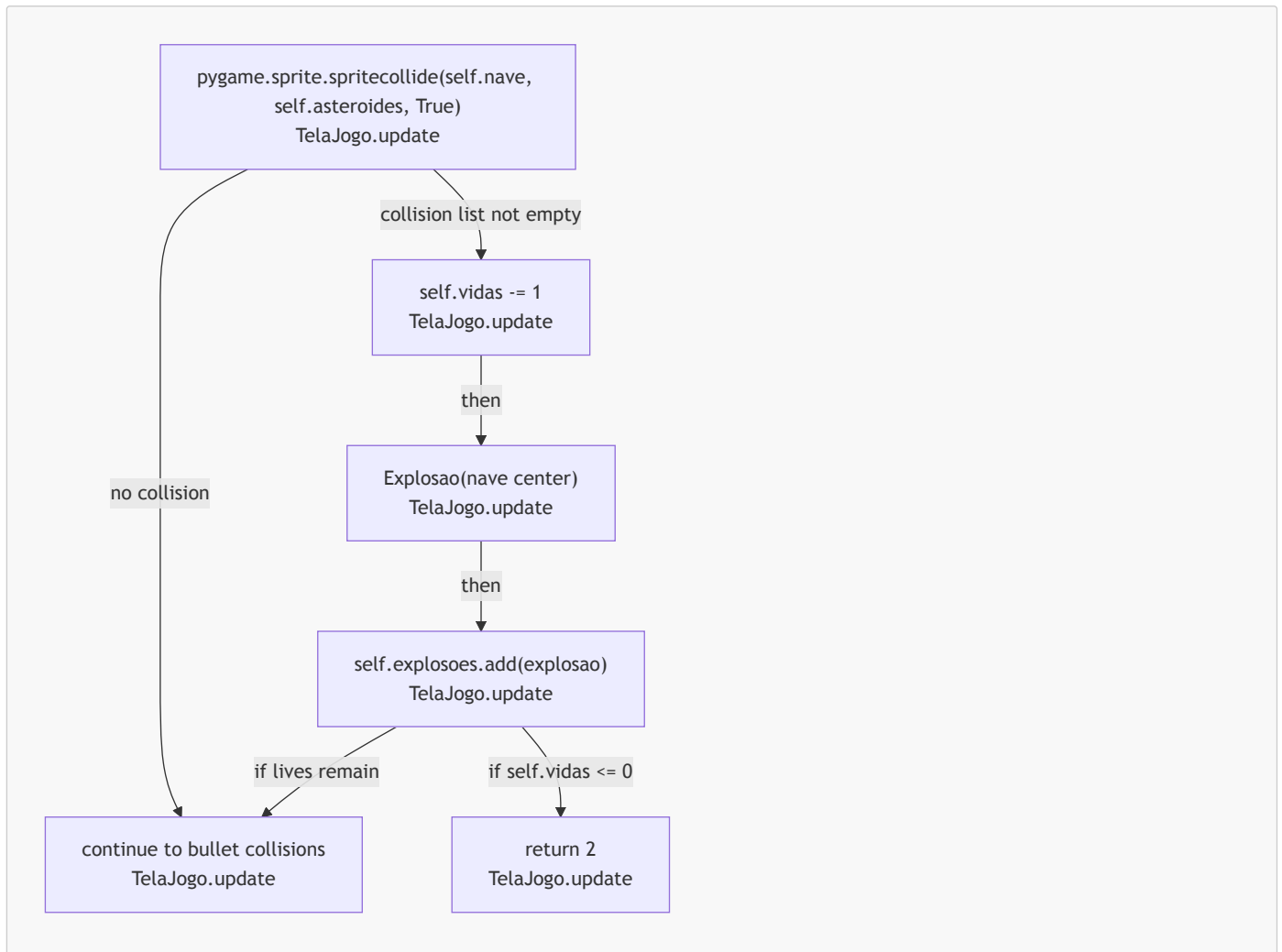
6.3 Player Primary Action Path



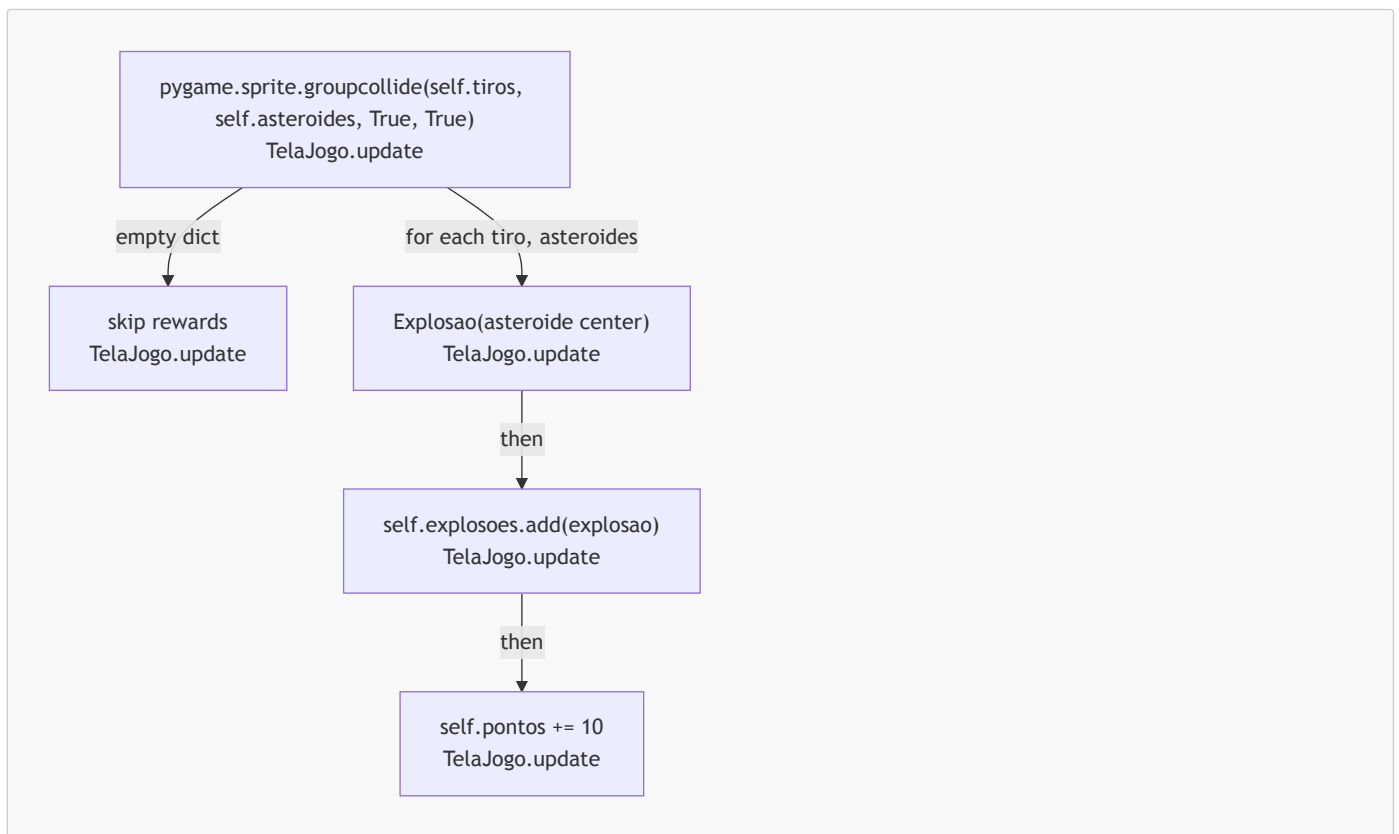
The primary action is shooting. A `KEYDOWN` event for Space plays `pew.wav`, creates a `Tiro` at the ship's top-center, adds it to the `tiros` group, and the draw path renders the group (`telas.py::TelaJogo.update`, `sprites.py::Tiro.__init__`, `sprites.py::Tiro.update`, `telas.py::TelaJogo.desenha`).

6.4 Collision Resolution Path

Ship versus asteroid:

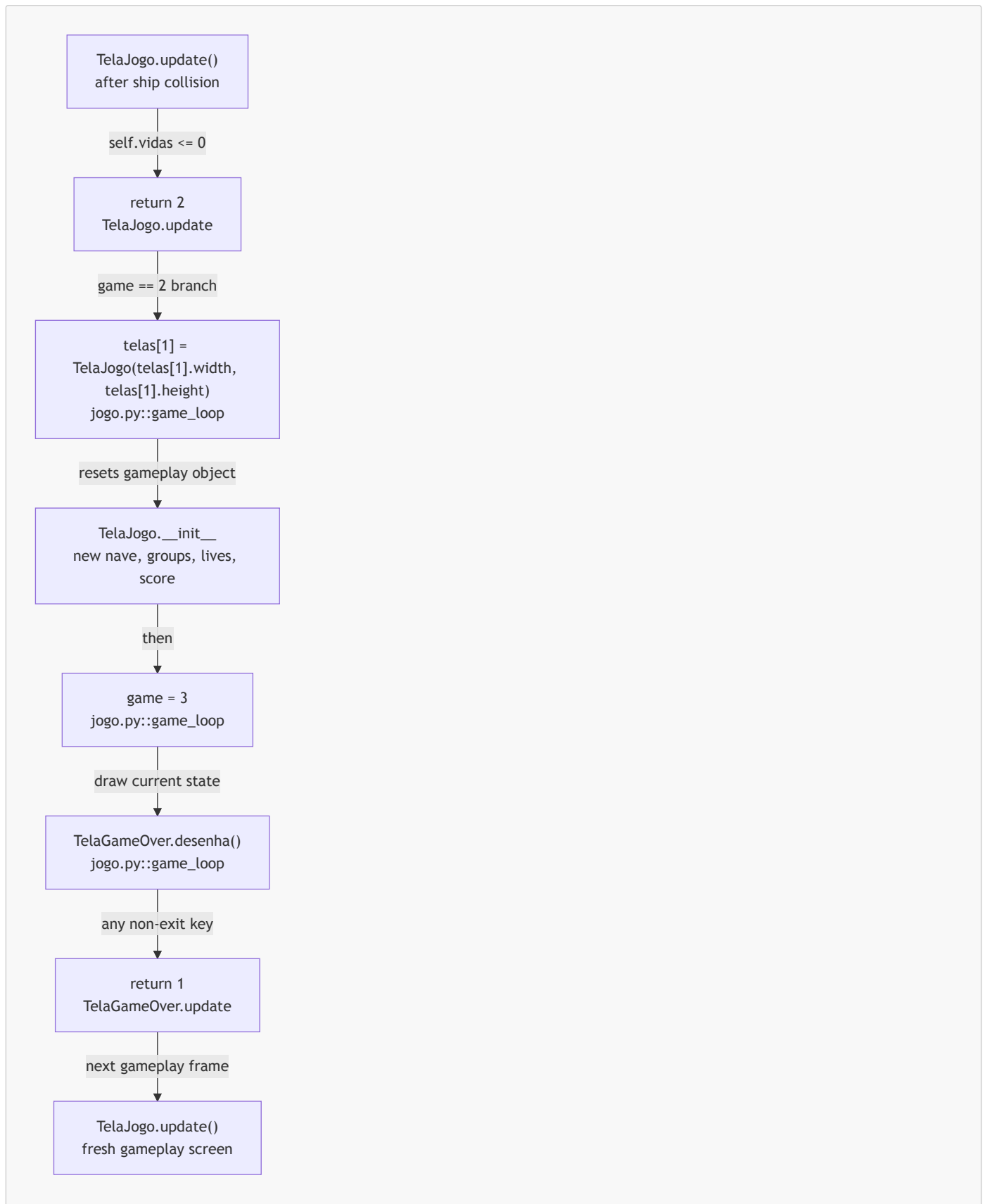


Bullet versus asteroid:



Ship collisions remove the asteroid, subtract one life, and create an explosion at the ship. Bullet collisions remove both bullet and asteroid, create an asteroid explosion, and award 10 points. Both collision categories use Pygame rectangle-based sprite collision helpers (`telas.py::TelaJogo.update`).

6.5 Level Transition / Respawn / Game Over Path



There is no level transition system and no in-place player respawn. The only reset path is game over: losing all lives reconstructs the entire `TelaJogo` object, then the game-over screen waits for a key to return to gameplay (`telas.py::TelaJogo.update`, `jogo.py::game_loop`, `telas.py::TelaGameOver.update`).

7. Game-Specific Dissection

7.1 Player Avatar

Topic	Observed behavior	Evidence
Class/module	Player is <code>Nave</code> in <code>sprites.py</code>	<code>sprites.py::Nave</code>
Construction path	<code>TelaJogo.__init__</code> creates <code>self.nave = Nave(self.width, self.height)</code>	<code>telas.py::TelaJogo.__init__</code>
Spawn location	<code>rect.center = (WIDTH // 2, HEIGHT - 100)</code> . With default <code>600 x 900</code> , inferred center is <code>(300, 800)</code>	<code>sprites.py::Nave.__init__</code> , <code>jogo.py::__main__</code>
Starting stats/defaults	<code>vel = 0.1</code> , all movement keys false, <code>t0 = pygame.time.get_ticks()</code>	<code>sprites.py::Nave.__init__</code>
Movement model	Event handlers set booleans; <code>update()</code> applies <code>x/y += vel * dt</code> , where <code>dt = 1000 / (t1 - t0)</code>	<code>sprites.py::Nave.keydown_event</code> , <code>sprites.py::Nave.keyup_event</code> , <code>sprites.py::Nave.update</code>
Bounds	If proposed x/y goes outside the screen, that axis reverts to its previous value	<code>sprites.py::Nave.update</code>
Animation	Not found; the ship uses one scaled image	<code>sprites.py::Nave.__init__</code> , <code>sprites.py::Nave.draw</code>
Primary ability	Space creates a <code>Tiro</code> at <code>nave.rect.centerx</code> , <code>nave.rect.top</code> and plays shot sound	<code>telas.py::TelaJogo.update</code>
Health/lives	Lives live on <code>TelaJogo</code> , not on <code>Nave</code> ; start at 3; collision subtracts 1; <code>vidas <= 0</code> causes game over	<code>telas.py::TelaJogo.__init__</code> , <code>telas.py::TelaJogo.update</code>
Invulnerability/cooldowns	Not found	<code>telas.py::TelaJogo.update</code>

Values are hardcoded rather than externally configurable. `vidas`, `pontos`, `level`, `number_of_asteroids`, and `key_state` mutate at runtime (`telas.py::TelaJogo.__init__`, `telas.py::TelaJogo.update`, `sprites.py::Nave.keydown_event`, `sprites.py::Nave.keyup_event`).

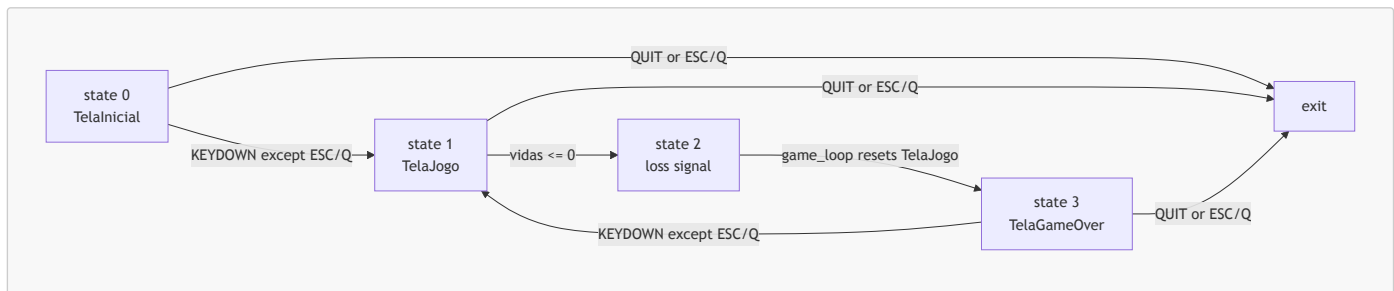
7.2 NPCs / Enemies / Interactive Entities

Name	Class	Asset(s)	Behavior model	Key stats	Spawn source
Asteroid	<code>Asteroides</code>	<code>assets/img/meteorBrown_med1.png</code>	Random spawn above screen; drifts down and sideways; killed below screen	Size <code>w=30..60</code> , <code>h=20..40</code> ; <code>vel_x=random.uniform(0.1,0.5)/40*level*±1</code> ; <code>vel_y=random.uniform(0.1,0.5)/10*level</code>	Initial 15 in <code>TelaJogo.__init__</code> ; replenished in <code>TelaJogo.update</code>
Bullet	<code>Tiro</code>	<code>assets/img/laserRed16.png</code>	Spawned on Space; moves upward; killed above screen	Scaled <code>(20, 40)</code> , <code>vel = 0.3</code> , <code>t0 = -1</code>	<code>TelaJogo.update</code> Space key
Explosion	<code>Explosao</code>	<code>assets/img/regularExplosion00.png</code> through <code>assets/img/regularExplosion08.png</code>	Temporary visual animation; advances every 50 ms; removed when finished	9 frames, scaled <code>(50, 50)</code> , <code>frame_rate = 50</code>	Created on ship-asteroid and bullet-asteroid collision

7.3 Levels / Scenes / Rooms

Identifier/name	Source type	Dimensions	Tilesets/backgrounds	Spawn points	Exits/transitions	Completion/failure
Initial menu	Hardcoded screen class	<code>600 x 900</code> default	Black background, font-rendered text	Not applicable	Non-exit key to gameplay; <code>QUIT</code> , <code>ESC</code> , or <code>Q</code> to exit	Not applicable
Gameplay	Hardcoded/procedural screen	<code>600 x 900</code> default	<code>starfield.png</code> scaled to screen plus 50 random	Ship at bottom-center;	<code>vidas <= 0</code> to game over; <code>QUIT</code> ,	Failure when <code>vidas <= 0</code> ; no win

Identifier/name	Source type	Dimensions	Tilesets/backgrounds	Spawn points	Exits/transitions	Completion/failure
			drawn stars	asteroids random above screen	ESC, or Q to exit	condition
Game over	Hardcoded screen class	600 x 900 default	Black background, font- rendered text	Not applicable	Non-exit key to gameplay; QUIT, ESC, or Q to exit	Not applicable



Level progression is not a scene transition. It is a numeric difficulty value: `level = int(self.pontos // 100) + 1`, and target asteroid count becomes `15 + self.level * 5` (`telas.py::TelaJogo.update`).

7.4 Commands & Input Map

Input	Event/API	Handler	Effect	Active state(s)	Evidence
Window close	<code>pygame.QUIT</code> from <code>pygame.event.get()</code>	All screen <code>update()</code> methods	Return <code>-1</code> ; main loop exits	Initial menu, gameplay, game over	<code>telas.py::TelaInicial.update</code> , <code>telas.py::TelaJogo.update</code> , <code>telas.py::TelaGameOver.update</code> , <code>jogo.py::game_loop</code>
ESC	<code>pygame.KEYDOWN</code> , <code>pygame.K_ESCAPE</code>	All screen <code>update()</code> methods	Return <code>-1</code> ; main loop exits	Initial menu, gameplay, game over	<code>telas.py::TelaInicial.update</code> , <code>telas.py::TelaJogo.update</code> , <code>telas.py::TelaGameOver.update</code>
Q	<code>pygame.KEYDOWN</code> , <code>pygame.K_q</code>	All screen <code>update()</code> methods	Return <code>-1</code> ; main loop exits	Initial menu, gameplay, game over	<code>telas.py::TelaInicial.update</code> , <code>telas.py::TelaJogo.update</code> , <code>telas.py::TelaGameOver.update</code>
Any other key	<code>pygame.KEYDOWN</code>	<code>TelaInicial.update</code>	Return <code>1</code> ; enter gameplay	Initial menu	<code>telas.py::TelaInicial.update</code>
Any other key	<code>pygame.KEYDOWN</code>	<code>TelaGameOver.update</code>	Return <code>1</code> ; restart gameplay	Game over	<code>telas.py::TelaGameOver.update</code> , <code>jogo.py::game_loop</code>
W press/release	<code>pygame.KEYDOWN / KEYUP</code> , <code>pygame.K_w</code>	<code>TelaJogo.update</code> delegates to <code>Nave.keydown_event /</code> <code>Nave.keyup_event</code>	Set/clear <code>key_state['UP']</code> ; ship moves upward during <code>Nave.update</code>	Gameplay	<code>telas.py::TelaJogo.update</code> , <code>sprites.py::Nave.keydown_event</code> , <code>sprites.py::Nave.keyup_event</code> , <code>sprites.py::Nave.update</code>
S press/release	<code>pygame.KEYDOWN / KEYUP</code> , <code>pygame.K_s</code>	Same as above	Set/clear <code>key_state['DOWN']</code> ; ship moves downward	Gameplay	<code>sprites.py::Nave.keydown_event</code> , <code>sprites.py::Nave.keyup_event</code> , <code>sprites.py::Nave.update</code>
A press/release	<code>pygame.KEYDOWN / KEYUP</code> , <code>pygame.K_a</code>	Same as above	Set/clear <code>key_state['LEFT']</code> ; ship moves left	Gameplay	<code>sprites.py::Nave.keydown_event</code> , <code>sprites.py::Nave.keyup_event</code> , <code>sprites.py::Nave.update</code>
D press/release	<code>pygame.KEYDOWN / KEYUP</code> , <code>pygame.K_d</code>	Same as above	Set/clear <code>key_state['RIGHT']</code> ; ship moves right	Gameplay	<code>sprites.py::Nave.keydown_event</code> , <code>sprites.py::Nave.keyup_event</code> , <code>sprites.py::Nave.update</code>
Space	<code>pygame.KEYDOWN</code> , <code>pygame.K_SPACE</code>	<code>TelaJogo.update</code>	Play shot sound and add a <code>Tiro</code>	Gameplay	<code>telas.py::TelaJogo.update</code> , <code>sprites.py::Tiro.__init__</code>
Held-key polling	<code>pygame.key.get_pressed</code>	Not found	Not applicable	Not applicable	All provided Python modules

Input	Event/API	Handler	Effect	Active state(s)	Evidence
Mouse events	Pygame mouse event APIs	Not found	Not applicable	Not applicable	All provided Python modules
Joystick/gamepad	Pygame joystick APIs	Not found	Not applicable	Not applicable	All provided Python modules

7.5 Asset & Media Map

Images/sprites:

Path/reference	Loaded by	Used by	Type	Notes	Evidence
assets/img/playerShip1_orange.png	Nave.__init__	Player ship	PNG	Definitely loaded; scaled to (50, 38)	sprites.py::Nave.__i
assets/img/meteorBrown_med1.png	Asteroides.__init__	Asteroids	PNG	Definitely loaded; scaled to random size	sprites.py::Asteroid
assets/img/laserRed16.png	Tiro.__init__	Bullet	PNG	Definitely loaded when a shot is constructed; scaled to (20, 40)	sprites.py::Tiro.__i
assets/img/regularExplosion0{i}.png	Explosao.__init__	Explosion animation	PNG sequence	Referenced via constructed path for <code>i = 0..8</code> ; files exist as <code>regularExplosion00.png</code> through <code>regularExplosion08.png</code>	sprites.py::Explosao
assets/img/starfield.png	TelaJogo.__init__	Gameplay background	PNG	Definitely loaded; scaled to window size	telas.py::TelaJogo._
assets/img/license.txt	Not loaded	Not used	Text	Asset metadata only	All provided Python modul

Audio/music:

Path/reference	Loaded by	Used by	Type	Notes	Evidence
assets/snd/tgfcoder-FrozenJam-SeamlessLoop.ogg	pygame.mixer.music.load in TelaJogo.__init__	Background music	Ogg/Vorbis	Definitely loaded; played looping with <code>pygame.mixer.music.play(-1)</code> once gameplay updates	telas.py::TelaJogo.__init telas.py::TelaJogo.update
assets/snd/pew.wav	pygame.mixer.Sound in TelaJogo.__init__	Shot sound	WAV	Definitely loaded; played on Space	telas.py::TelaJogo.__init telas.py::TelaJogo.update
assets/snd/expl3.wav	Not loaded	Not used	WAV	Suspicious unused explosion sound	All provided Python modules
assets/snd/expl6.wav	Not loaded	Not used	WAV	Suspicious unused explosion sound	All provided Python modules

Fonts:

Path/reference	Loaded by	Used by	Type	Notes	Evidence
assets/font/PressStart2P.ttf	All screen constructors	Menu, game-over, HUD text	TTF	Definitely loaded at size 20	telas.py::TelaInicial.__init__, telas.py::TelaJogo.__init__, telas.py::TelaGameOver.__init__
assets/font/LICENSE.txt	Not loaded	Not used	Text	Font metadata only	All provided Python modules
assets/font/FONTLOG.txt	Not loaded	Not used	Text	Font metadata only	All provided Python modules

Data/config files:

Path/reference	Loaded by	Used by	Type	Notes	Evidence
Data/config files	Not found	Not applicable	Not applicable	No JSON, CSV, TMX, INI, TOML, or save/config loading is present	All provided Python modules

7.6 HUD, UI, Menus, and Overlays

Element	Draw/update location	Data source	Active states	Evidence
Initial menu text	<code>TelaInicial.desenha</code>	Hardcoded strings "Pressione qualquer", "tecla para começar", "Q para sair"	Initial menu	<code>telas.py::TelaInicial.desenha</code>
Game-over text	<code>TelaGameOver.desenha</code>	Hardcoded strings "Game Over", "Pressione qualquer tecla", "para jogar novamente", "Q para sair"	Game over	<code>telas.py::TelaGameOver.desenha</code>
Lives background hearts	<code>TelaJogo.desenha</code>	<code>self.coracoes * self.max_vidas</code> , drawn white	Gameplay	<code>telas.py::TelaJogo.desenha</code>
Current lives hearts	<code>TelaJogo.desenha</code>	<code>self.coracoes * self.vidas</code> , drawn red	Gameplay	<code>telas.py::TelaJogo.desenha</code>
FPS text	<code>TelaJogo.desenha</code>	<code>fps = 1000 / (t1 - self.t0)</code>	Gameplay	<code>telas.py::TelaJogo.desenha</code>
Score text	<code>TelaJogo.desenha</code>	<code>int(self.pontos)</code>	Gameplay	<code>telas.py::TelaJogo.desenha</code>
Pause overlay	Not found	Not applicable	Not applicable	All provided Python modules
Dialog/tutorial text beyond menus	Not found	Not applicable	Not applicable	All provided Python modules
Debug overlays beyond FPS	Not found	Not applicable	Not applicable	All provided Python modules

7.7 Scoring, Progression & Persistence

Destroying an asteroid with a bullet adds **10** points (`telas.py::TelaJogo.update`). The game also adds `self.fps / 1000` to `pontos` every gameplay update, where `self.fps` is calculated in `TelaJogo.desenha` as `1000 / (t1 - self.t0)` (`telas.py::TelaJogo.update`, `telas.py::TelaJogo.desenha`).

Progression is score-based but not scene-based. `level = int(self.pontos // 100) + 1`, and `number_of_asteroids = 15 + self.level * 5` (`telas.py::TelaJogo.update`). Asteroid speed can scale with `level`, but only when `Asteroides` receives the level argument; one replenishment loop passes `self.level`, and a later loop does not, so some replacement asteroids use the default `level = 1` (`telas.py::TelaJogo.update`, `sprites.py::Asteroides.__init__`).

Persistence/save/load/high-score/profile behavior is absent. No serialization schema, storage path, file format, or save/load root exists in the provided Python files.

8. Collision & Physics Model

Collision detection uses Pygame sprite rectangle helpers. Ship-asteroid collisions use `pygame.sprite.spritecollide(self.nave, self.asteroides, True)`, which removes collided asteroids. Bullet-asteroid collisions use `pygame.sprite.groupcollide(self.tiros, self.asteroides, True, True)`, which removes both bullets and asteroids (`telas.py::TelaJogo.update`). No masks, layers, custom hitboxes, bounce physics, gravity, friction, knockback, or cooldown timers are present.

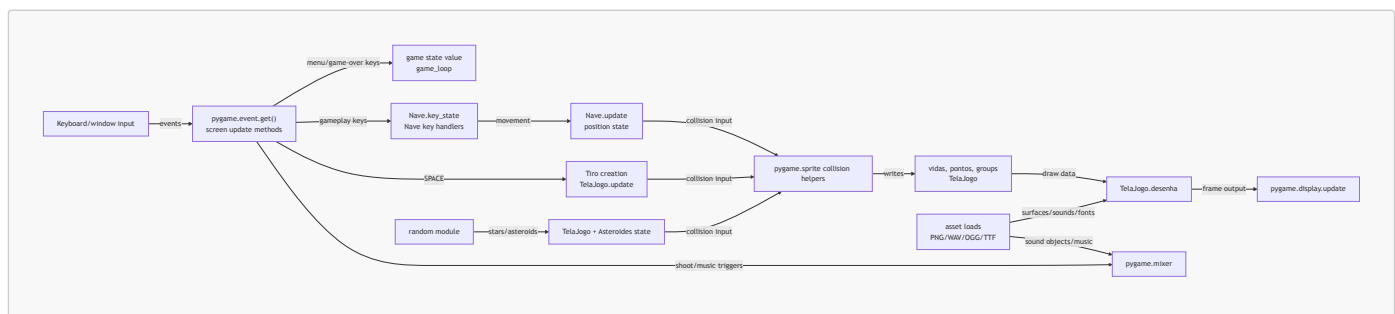
The coordinate system follows normal Pygame screen coordinates: positive x moves right and positive y moves down. Movement is manual rectangle mutation. The code attempts time-based movement but uses inverse elapsed milliseconds, `dt = 1000 / (t1 - self.t0)`, so movement depends on frame timing in a nonstandard way and risks division by zero if two updates happen in the same millisecond (`sprites.py::Nave.update`, `sprites.py::Asteroides.update`, `sprites.py::Tiro.update`).

Collider A	Collider B	Detection method	Resolution/effect	Evidence
Nave	Asteroides group	<code>pygame.sprite.spritecollide(self.nave, self.asteroides, True)</code>	Remove collided asteroid(s), subtract 1 life, create ship explosion, return game-over signal if lives reach zero	<code>telas.py::TelaJogo.update</code>
Tiro group	Asteroides group	<code>pygame.sprite.groupcollide(self.tiros, self.asteroides, True, True)</code>	Remove bullet and asteroid, create asteroid explosion, add 10 points	<code>telas.py::TelaJogo.update</code>
Asteroides	Bottom of screen	<code>if self.rect.y > self.HEIGHT</code>	<code>self.kill()</code>	<code>sprites.py::Asteroides.update</code>
Tiro	Top of screen	<code>if self.rect.y < -self.rect.height</code>	<code>self.kill()</code>	<code>sprites.py::Tiro.update</code>
Nave	Screen bounds	Proposed coordinate checks against <code>0, WIDTH - rect.width, HEIGHT - rect.height</code>	Revert movement on the axis that would leave bounds	<code>sprites.py::Nave.update</code>

Exact constants:

Value	Meaning	Evidence
600, 900	Default width and height	jogo.py::__main__
60	Main loop FPS cap	jogo.py::game_loop
0.1	Player movement multiplier	sprites.py::Nave.__init__
0.3	Bullet movement multiplier	sprites.py::Tiro.__init__
3	Starting and max lives	telas.py::TelaJogo.__init__
15	Initial asteroid target count	telas.py::TelaJogo.__init__
50	Random star count and explosion frame delay in ms, in separate contexts	telas.py::TelaJogo.__init__, sprites.py::Explosao.__init__
10	Points per asteroid destroyed by bullet	telas.py::TelaJogo.update

9. Data Flow & Control Flow



Control roots:

Root type	Location	Notes
Entry point	jogo.py::__main__	Initializes Pygame, constructs screens, calls <code>game_loop</code>
Per-frame root	jogo.py::game_loop	Calls active screen <code>update()</code> , handles special states, calls active screen <code>desenha()</code> , ticks clock
Gameplay frame root	telas.py::TelaJogo.update and telas.py::TelaJogo.desenha	Update and render are split but both mutate gameplay-adjacent state; <code>desenha()</code> updates FPS and explosions
State transition roots	TelaInicial.update, TelaJogo.update, TelaGameOver.update, plus game_loop handling of 2 and -1	Numeric return values drive state transitions
Save/load roots	Not found	No persistence code exists

Persistence side channel: not applicable. Asset loading is direct at construction time; there is no resource cache layer. Randomness is used for stars and asteroids. Timing is read through `pygame.time.get_ticks()` and `pygame.time.Clock(jogo.py::game_loop, telas.py::TelaJogo.__init__, sprites.py::Asteroides.__init__, sprites.py::Nave.update, sprites.py::Asteroides.update, sprites.py::Tiro.update)`.

10. Code Quality & Engineering Risk Notes

Severity	Risk	Evidence
High	Movement and FPS calculations use <code>1000 / (t1 - t0)</code> , which can divide by zero and makes motion inversely proportional to elapsed time.	sprites.py::Nave.update, sprites.py::Asteroides.update, sprites.py::Tiro.update, telas.py::TelaJogo.desenha
High	Normal gameplay returns <code>True</code> , and <code>game_loop</code> uses it as a dictionary key. This works only because <code>True == 1</code> in Python, creating fragile coupling between booleans and numeric screen IDs.	telas.py::TelaJogo.update, jogo.py::game_loop, jogo.py::__main__
Medium	TelaJogo owns many unrelated responsibilities: input, entities, collision, score, difficulty, audio, HUD, rendering, and explosion updates.	telas.py::TelaJogo.__init__, telas.py::TelaJogo.update, telas.py::TelaJogo.desenha
Medium	Asset paths are hardcoded relative paths, so running from another working directory can break asset loading.	sprites.py::Nave.__init__, sprites.py::Asteroides.__init__, sprites.py::Tiro.__init__, sprites.py::Explosao.__init__, telas.py::*.__init__
Medium	There is no exception handling or validation for missing images, sounds, music, or fonts.	sprites.py::*.__init__, telas.py::*.__init__

Severity	Risk	Evidence
Medium	Asteroid difficulty scaling is inconsistent: one replenishment loop passes <code>self.level</code> , while a later replenishment loop uses default <code>level=1</code> .	<code>telas.py::TelaJogo.update</code> , <code>sprites.py::Asteroides.__init__</code>
Medium	<code>self.musica_fundo</code> first stores the return value of <code>pygame.mixer.music.load</code> , then is overwritten as a boolean flag.	<code>telas.py::TelaJogo.__init__</code> , <code>telas.py::TelaJogo.update</code>
Low	<code>funcoes.py</code> is empty but imported, creating dead import noise.	<code>funcoes.py</code> , <code>jogo.py::__main__</code>
Low	Explosion sound files exist but are unused.	<code>telas.py::TelaJogo.__init__</code> , all provided Python modules
Low	<code>pygame.display.set_mode</code> is called in every screen constructor, and all screens are constructed at startup. This is redundant and makes display ownership implicit.	<code>jogo.py::__main__</code> , <code>telas.py::TelaInicial.__init__</code> , <code>telas.py::TelaJogo.__init__</code> , <code>telas.py::TelaGameOver.__init__</code>
Low	Magic numbers and hardcoded strings are scattered instead of named constants.	<code>jogo.py::__main__</code> , <code>jogo.py::game_loop</code> , <code>telas.py::TelaJogo.__init__</code> , <code>telas.py::TelaJogo.update</code> , <code>sprites.py:*.__init__</code>

Circular dependencies: not found. `jogo.py` imports `telas.py`, and `telas.py` imports `sprites.py`; `sprites.py` does not import back into `telas.py` or `jogo.py` (`jogo.py::__main__`, `telas.py::__module__`, `sprites.py::__module__`).

11. Gaps & Assumptions

11.1 Unknowns

Unknown	Why it is unknown
Exact Pygame version	No dependency file is present, and the local environment did not have importable <code>pygame</code> .
Python version target	No <code>pyproject.toml</code> , <code>requirements.txt</code> , shebang, or runtime marker is present.
Intended game title or final design	README is placeholder-like; code caption is only " <code>Jogo Pygame</code> ".
Whether unused explosion sounds were intended	<code>expl3.wav</code> and <code>expl6.wav</code> exist but are never loaded.
Runtime behavior on the original developer machine	Static code can be read, but the local environment lacks Pygame, so runtime was not executed.
Intended balance for movement, scoring, and level scaling	Numeric formulas are present, but no design notes explain intent.
Persistence/high-score plans	No save/load code or data files are present.

11.2 Inferences

Inference	Basis
The state system is inferred rather than formal.	State is represented by integer return values and dictionary keys, with no enum or state object (<code>jogo.py::game_loop</code> , <code>jogo.py::__main__</code>).
The gameplay state effectively remains <code>1</code> because <code>True</code> indexes the same dictionary entry as <code>1</code> .	<code>TelaJogo.update</code> returns <code>True</code> ; <code>game_loop</code> assigns that to <code>game</code> and indexes <code>telas[game]</code> (<code>telas.py::TelaJogo.update</code> , <code>jogo.py::game_loop</code>).
The game is an endless survival shooter.	There is no win condition or level scene transition; <code>level</code> only changes asteroid count and sometimes speed (<code>telas.py::TelaJogo.update</code>).
State <code>2</code> is not a drawable screen.	<code>telas[2]</code> is <code>''</code> , and <code>game_loop</code> intercepts <code>game == 2</code> before drawing (<code>jogo.py::__main__</code> , <code>jogo.py::game_loop</code>).
Score includes a survival/time component, but it is implemented through FPS rather than elapsed time.	<code>pontos += self.fps / 1000</code> , and <code>self.fps</code> is computed during drawing (<code>telas.py::TelaJogo.update</code> , <code>telas.py::TelaJogo.desenha</code>).